# Lesson 3B Reference: f-Strings, Escapes & Multiline Text

Everything taught in class today, on one page. Keep it — you'll use f-strings in every unit from here on.

**You built:** the Match Recap Card ( `lesson3B_recap_starter.py` )

---

## Quick translation table

| JavaScript | Python |
|---|---|
| `` `Hi ${name}` `` | `f"Hi {name}"` |
| `` `${a} and ${b}` `` | `f"{a} and {b}"` |
| `\n` `\t` `\\` | identical — no change |
| backticks spanning lines | `'''` or `"""` |

### From Lesson 3A — still needed today

The recap card also uses these:

```
title = "MATCH RECAP - VALORANT"

print("a", "b", sep=" | ")   # sep: what goes between
print("x", end="")           # end: what goes after
print("=" * 40)              # repeat a string
padding = (40 - len(title)) // 2
```

---

## f-strings

The direct equivalent of a JavaScript template literal.

```
name = "Alex"
age = 15

print(f"Hi {name}, you are {age}")   # Hi Alex, you are 15
```

Without them you'd be doing this:

```
print("Hi " + name + ", you are " + str(age))
```

Quote juggling, plus `str()` on every number — Python won't glue a number onto a string with `+` the way JavaScript will.

Two differences from JS:

- Backticks become **normal quotes with an `f` in front**
- `${ }` becomes just `{ }`

## ⚠️ The trap: the missing f

```
print("Hi {name}")     # Hi {name}   ← forgot the f
print(f"Hi {name}")    # Hi Alex
```

**No error appears.** It just prints the braces. If you see `{ }` in your output, you left off an `f`. That's the whole diagnosis.

## Expressions work inside the braces

```
age = 15
price = 24.99
qty = 3
name = "Alex"

print(f"Next year you turn {age + 1}")   # Next year you turn 16
print(f"Total: {price * qty}")           # Total: 74.97
print(f"Loud: {name.upper()}")           # Loud: ALEX
```

Any expression is allowed — arithmetic, method calls, comparisons.

> If you need quotes *inside* the braces, use the other kind:
>
> `f"Item: {'Headphones'.upper()}"`

## Calculating inside an f-string

This is how the accuracy bar in the recap card works:

```
accuracy = 82
filled = accuracy // 10        # 8
empty = 10 - filled            # 2

print(f"Accuracy: {'#' * filled}{'.' * empty} {accuracy}%")
# Accuracy: ########.. 82%
```

---

# Escape sequences

Identical to JavaScript. Nothing new to learn.

| Sequence | Meaning |
|---|---|
| `\n` | New line |
| `\t` | Tab |
| `\'` | A single quote |
| `\"` | A double quote |
| `\\` | One real backslash |

```
print("Welcome\nto\n\nPython!")
# Welcome
# to
#
# Python!

print("Name\tAge\tGrade")          # Name    Age     Grade
print("Path: C:\\Users\\Student") # Path: C:\Users\Student
```

Two `\n` in a row make a blank line. Two backslashes make one — the first escapes the second.

---

# Triple quotes

They do **one** job: let a string span multiple lines and hold both quote types without escaping.

```
print("""He said "that's mine" """)
```

That's the cleaner answer to the problem left open at the end of Lesson 3A.

```
notes = '''Map: Ascent
Duration: 42 min
Both " and ' are fine in here'''

print(notes)
```

Every line break you type is preserved in the output. `'''` and `"""` work identically.

## ⚠️ The trap: triple quotes are NOT template literals

This is the single most common mistake coming from JavaScript.

```
name = "Alex"

print("""Hi {name}""")    # Hi {name}   ← prints the braces
print(f"""Hi {name}""")   # Hi Alex     ← the f does the work
```

| | Spans lines? | Substitutes variables? |
|---|---|---|
| `"""..."""` | Yes | **No** |
| `f"..."` | No | Yes |
| `f"""..."""` | Yes | Yes |

In JavaScript, backticks do both jobs at once. In Python they're separate tools — combine them with `f"""` when you need both.

```
game_map = "Ascent"
minutes = 42

print(f"""
Match Notes:
\tMap: {game_map}
\tDuration: {minutes} min""")
```

---

# Splitting a long line

```
message = ("This string is long, so we "
           "split it across two lines")
```

Two strings side by side inside parentheses join automatically.

> You may see `\` used at the end of a line for this. Avoid it — a single invisible space after the backslash breaks it, and PEP 8 prefers parentheses.

---

# One you'll see in old code

```
name = "Alex"

print("Hi {}".format(name))   # the old way
print(f"Hi {name}")           # the way we use
```

`.format()` came before f-strings. Recognize it in older tutorials and StackOverflow answers; don't write it.

---

# Common errors

| Error or symptom | Cause | Fix |
|---|---|---|
| Output shows `{name}` literally | Missing the `f` prefix | Add `f` before the opening quote |
| Output shows `{name}` from a `"""` string | Triple quotes don't substitute | Use `f"""` instead |
| `SyntaxError` inside an f-string | Same quote type inside the braces as outside | Use the other kind inside |
| `TypeError: can only concatenate str` | Used `+` with a number | Use an f-string, or wrap in `str()` |
| Tabs look wrong in output | Terminal tab width varies | Normal — `\t` isn't a fixed number of spaces |
| Bar or border wrong width | Typed it by hand instead of calculating | Use `//` and the `*` operator |

---

# Checklist before you push

- ☐ Header comment at the top: your name, assignment, date
- ☐ Every `{ }` in your output is filled — no stray braces
- ☐ `snake_case` variable names
- ☐ The file **runs** without errors
- ☐ Output matches the target exactly, including tabs and blank lines
- ☐ Bars and padding are **calculated**, not typed by hand
- ☐ Committed **and pushed** — check github.com to confirm

---

**Next lesson:** variables and types — no `let`, no `const`, no declared types.